

Оптимальное ML-решение задачи «восстановление изображения + сборка пазла» (сетка 24×24 , метрика mean SSIM)

TL;DR

- **Задача почти-разделима на два подпроблема** — сборку 576 фрагментов в известную сетку 24×24 и честное восстановление качества — и оба надо решать сильно и мерить независимо. Главный архитектурный вывод по сборке: замените siamese/dot-product скорер (потолок recall@1 ≈ 0.16) на **early-fusion cross-scorer, который видит шов обоих фрагментов совместно** (архитектура типа JigsawNet с RoIAlign на стыке), используя siamese лишь как быстрый ретривер для шортлиста $\rightarrow$ «retrieve-then-rerank».
- **Восстановление — крупнейший дополнительный рычаг SSIM** (по вашей калибровке +0.15...+0.35 сверх идеальной сборки). Обучайте сеть класса **NAFNet/Restormer** на неограниченных синтетических парах, воспроизводящих реверс-инжинирный деградатор (affine $\rightarrow$ шум $\sigma 40-55 \rightarrow$ блюр $3 \times 3 \rightarrow$ JPEG q35-50), с лоссом **MS-SSIM+L1**, и восстанавливайте **полное собранное изображение целиком** (а не по фрагментам), чтобы сеть сама чинила швы. Adversarial/перцептивные лоссы под метрику SSIM применять НЕЛЬЗЯ (perception-distortion tradeoff понижает SSIM).
- **Оптимальный порядок операций:** пофрагментная фотометрическая нормализация $\rightarrow$ эмбединг + cross-rescoring $\rightarrow$ сборка с жёсткой эксплуатацией сетки 24×24 (loop-constraints + линейное назначение) $\rightarrow$ полнокадровое восстановление с TLC $\rightarrow$ x8 geometric self-ensemble. Один цикл итеративного refinement (solve $\rightarrow$ restore $\rightarrow$ re-solve) даёт заметный прирост. Все шаги — честная реставрация; эксплойт монотонных квадратов (~ 0.4 SSIM) отвергается сознательно.

Ключевые выводы

1. Разделимость и калибровка потолков (фундамент стратегии)

Ваша калибровка задаёт карту рычагов:

Сценарий	mean SSIM
Identity (без сборки/реставрации)	0.08–0.11
Идеальная сборка, без реставрации	0.43–0.50
Идеальная сборка + реставрация	0.6–0.8
Текущий лидер	0.40

Вывод: даже безупречная сборка без реставрации уже обходит лидера, а реставрация добавляет ещё 0.15–0.35. Поскольку подзадачи почти-разделимы (сборка задаёт геометрию, реставрация — фотометрию/детали), их следует разрабатывать и валидировать как **независимо измеримые модули**, и бюджет усилий делить: сначала — гарантировать высокую точность сборки (иначе реставратор чинит перемешанный мусор), затем — вытягивать реставрацию.

2. Часть А — Сборка пазла: state of the art

Классические метрики совместимости краёв. Опорная метрика — **Mahalanobis Gradient Compatibility (MGC, Gallagher, 2012)**: штрафует изменения градиентов интенсивности на стыке (а не сами интенсивности) и обучает ковариацию цветовых каналов через махаланобисово расстояние; добавляются «dummy-градиенты» для численной устойчивости. По бенчмарку Dirauf et al. (2025, arXiv:2507.07828): «Gallagher shows that MGC significantly outperforms previous compatibility metrics which rely on pixel intensity differences at piece boundaries. This is also confirmed by Giuliari et al. and Son et al.» Вторая сильнейшая метрика — **L1-норма Paikin & Tal (CVPR 2015)**: они заменяют prediction-based норму Pomeranz $(L_p)^q$ ($p=3/10, q=1/16$) на L1 («L1 — компромисс между L_p -нормами») плюс **асимметричная dissimilarity** $D(p_i, p_{j, right}) \neq D(p_j, p_{i, left})$ и стратегический выбор якорного фрагмента. **Pomeranz et al. (2011)** ввели prediction-based dissimilarity (предсказание значений на границе через разложение Тейлора; оптимальная L_p -норма при $p \approx 0.3$, точность ~86%) и концепцию **best-buddies** (два фрагмента — взаимно наиболее совместимые). **Cho et al. (2010)** — loopu belief propagation на графовой модели. (arxiv + 2)

Критический момент для нашей задачи: метрики на попиксельных значениях/градиентах деградируют при зашумлённых/JPEG/блюр-краях. Бенчмарк Dirauf et al. показывает крутое падение эвристик при eroded edges/contents — «Corruptions affecting pixel values ... lead to a particularly sharp decline, as they directly impact the pairwise compatibility metric.» Отсюда: **нельзя полагаться на сырые MGC/L1 по деградированным краям — нужен обучаемый скорер, натренированный на нашей деградации**, либо MGC следует считать по уже фотометрически-нормализованным (в идеале — предварительно денойзенным) краям.

Алгоритмы сборки при известной попарной матрице.

- **Greedy tree-based** (Gallagher): «constrained minimum spanning tree», жадно сливает компоненты с соблюдением геометрической согласованности.
- **Loop constraints** (Son, Hays, Cooper, ECCV 2014): ищет 4-циклы фрагментов как форму outlier-rejection, агрегирует «петли петель» снизу-вверх; снижение ошибки реконструкции до 70% на сложнейшем типе. Есть иерархическая версия (hierarchical loop constraints).
(arXiv)
- **Linear programming** (Yu, Russell, Agapito, 2015): эксплуатирует все попарные матчи одновременно; **самый робастный к коррупции** в бенчмарке Dirauf et al.
- **Genetic algorithm** (Sholomon et al.): для очень больших пазлов, кроссовер на best-buddies.
- **Relaxation labeling** (многофазный, Andaló-стиль преобразование dissimilarity→compatibility).

Эксплуатация известной сетки 24×24 — главный рычаг сборки. У нас нет вращений, нет пропусков/лишних, известен размер фрагмента 20×20 и точная сетка 24×24. Это переводит задачу из общего NP-hard в жёстко-структурированную: позиции — целочисленная 2D-решётка, loop closures (4-циклы) формулируются тривиально, а глобальная согласованность обеспечивается **линейным назначением (венгерский алгоритм / Hungarian, $O(n^3)$)** фрагментов на 576 фиксированных ячеек по стоимости, агрегирующей попарные совместимости.

Глубокое обучение (ключ к преодолению вашего бутылочного горлышка).

- **DNN-Buddies** (Sholomon, David, Netanyahu, ICANN 2016, arXiv:1711.08762): первая нейросетевая estimation-метрика. FFNN из 5 полносвязных слоёв (336→100→100→100→2) на **конкатенированных пограничных полосах** (по 2 столбца пикселей с каждого края в YUV). Обучение с **informed hard-negative undersampling**: для каждого края берётся наиболее и второй-по-совместимости сосед; near-miss (визуально похожие, но неверные) идут в жёсткие негативы. Precision (verbatim): «Using the well known dataset presented by Cho et al. of 20 432-piece puzzles, we obtained a precision of 94.83%.» Интеграция в GA-solver подняла neighbor-accuracy до 95.65%/96.37%/95.86% на 432/540/805-piece.
(arxiv + 2)
- **JigsawNet** (Canyu Le & Xin Li, IEEE TIP 2019, vol. 28, pp. 4000–4015, arXiv:1809.04137): CNN-классификатор совместимости, который **видит стык (стичинг-регион) двух фрагментов совместно**, с RoIAlign на шве (перенос внимания на границу) и AdaBoost для борьбы с дисбалансом. Precision вырос с 13.7% до 77.9% на сложнейшем наборе (TestingSet2) при сохранении recall ~96%. Это — эмпирическое доказательство превосходства early-fusion над попиксельными метриками. (arxiv) (arxiv)
- **Positional Diffusion** (Giuliani et al.): attention-based GNN, предсказывает нормализованные координаты фрагментов через диффузионный процесс; SOTA среди DL-решателей и, по Dirauf et al., **лучше всех адаптируется к коррупции при дообучении.**
- **Gumbel-Sinkhorn networks** (Mena et al., ICLR 2018, arXiv:1802.08665): дифференцируемая релаксация перестановок через оператор Синкхорна (итеративная

строчно-столбцовая нормализация → дважды-стохастическая матрица), позволяет обучать перестановку end-to-end. Полезно как опциональный глобальный «доводчик».

Разрешение бутылочного горлышка siamese (главный технический инсайт). Ваш siamese/dot-product скорер — это **bi-encoder (late fusion)**: каждый фрагмент кодируется независимо, сравнение — скалярным произведением. Он архитектурно менее выразителен, чем **cross-encoder (early fusion)**, который видит оба объекта совместно с полным вниманием. Аналогия из retrieval-литературы количественна: Poly-encoders (Humeau et al., ICLR 2020, arXiv:1905.01969) — на ConvAI2 «the Bi-encoder reaches 81.7% R@1 ... while the Cross-encoder achieves higher scores of 84.8% R@1», а при Reddit-претрейне $bi \approx 84.8\%$ против $cross \approx 87.9\%$ R@1 (разрыв ~3 пункта). Ваш PairwiseNet, видящий полосу (3,20,40) поперёк шва → логит, — это ровно cross-scorer, поэтому он «far more expressive». **Практический паттерн — retrieve-then-rerank**: siamese даёт быстрый шортлист top-K соседей (задаёт потолок recall — его надо поднять hard-negative-майнингом и большим контекстом шва), а cross/seam-сеть точно реранжирует top-K. Так вы получаете точность cross-encoder при приемлемой стоимости (перанжировать надо ~top-50 из 575, а не все N^2). (arXiv) (OpenReview)

3. Часть В — Восстановление: state of the art

Архитектуры и трейдоффы для одной GPU 8-16 ГБ.

- **NAFNet (Chen et al., ECCV 2022, arXiv:2204.04676)**: SimpleGate вместо нелинейных активаций. Verbatim: «33.69 dB PSNR on GoPro (for image deblurring), exceeding the previous SOTA 0.38 dB with only 8.4% of its computational costs; 40.30 dB PSNR on SIDD (for image denoising), exceeding the previous SOTA 0.28 dB with less than half of its computational costs.» **Оптимальный выбор для бюджетного трека** — лёгкий, быстрый, отличный на denoise+deblur. (arXiv)
- **Restormer (Zamir et al., CVPR 2022, arXiv:2111.09881)**: MDTA (attention по каналам, линейная сложность, не бьёт изображение на окна) + GDFN, 4-уровневый encoder-decoder. Лучший почти во всех задачах кроме SR. **Выбор для сильного трека** (если есть сервер помощнее).
- **SwinIR/HAT** — лучшие в SR, но тяжелее; **Uformer** — силён на denoise/deblur. Для нашей комбинированной деградации без даунсемплинга SR-специфика не нужна.

Где восстанавливать — решение. Восстанавливать **полное собранное изображение 480×480 целиком, после сборки**. Аргументы: (а) сеть учится чинить швы между фрагментами (границы 20-пиксельных плиток) и фотометрические скачки; (б) деградация пространственно-варьирующаяся (каждая плитка имеет свой шум/яркость) — полнокадровая сеть с достаточным рецептивным полем справляется с этим лучше, чем пофрагментная. Пофрагментное восстановление ДО сборки полезно только как вспомогательный шаг для улучшения матчинга краёв в итеративном refinement (см. Часть С), но финальный выход — полнокадровый. **TLC (Test-time Local Converter, Chen et al., ECCV 2022)**: устраняет train-test несоответствие статистик при patch-обучении и full-image инференсе — оборачивайте

глобальные операции (пулинг, нормализацию) в локальные на инференсе; прирост без дообучения. Используйте NAFNetLocal/Restormer+TLC.

Лоссы под SSIM. Основной — **MS-SSIM+L1 (Zhao et al., 2017, IEEE TCI)**: MS-SSIM сохраняет контраст в высокочастотных областях, L1 сохраняет цвета/яркость (равномерно взвешивает ошибку). Каноническая формула: $L = \alpha \cdot L_{\text{MS-SSIM}} + (1-\alpha) \cdot (G_{\sigma} \cdot L1)$, $\alpha \approx 0.84$, где L1 взвешивается гауссовым окном под MS-SSIM. **Критично:** метрика соревнования — SSIM (win_size=7, data_range=255, channel_axis=2), поэтому добавляйте прямой **SSIM-лосс с win_size=7** как компонент, чтобы выровнять обучение с метрикой. **Perception-distortion tradeoff (Blau & Michaeli, CVPR 2018)** доказывает: снижение искажения обязательно повышает вероятность различения от реальных изображений → GAN/перцептивные (LPIPS/adversarial) лоссы **понижают SSIM**. Это прямое разрешение вашего напряжения «честная реставрация vs метрика SSIM»: под SSIM НЕ используйте adversarial/LPIPS; честная реставрация здесь = регрессия к чистому оригиналу под MS-SSIM+L1, что и максимизирует SSIM без гейминга. (arXiv) (TheCVF)

Синтез обучающих пар. Реверс-инжинирный пайплайн (affine bright/contrast → +Gaussian noise $\sigma 40-55$ → Gaussian blur 3×3 → JPEG q35-50 пофрагментно; ΔSSIM синтетики к реальной ≈ 0.03) даёт **неограниченные пары с идеальными метками**. Из Real-ESRGAN (Wang et al., ICCV 2021 W) заимствуйте идею **domain randomization / рандомизации порядка и границ параметров** (high-order degradation) для робастности: слегка варьируйте порядок шум↔блюр, расширяйте диапазоны на $\pm 10-15\%$, добавляйте лёгкую вероятность двойного JPEG. **FBCNN (Jiang et al., ICCV 2021, arXiv:2109.14573)** — blind JPEG с предсказанием quality factor и QF-attention; полезен как компонент/инициализация для JPEG-специфичной части, т.к. JPEG q35-50 — доминирующий артефакт (виден 8×8 grid).

Фотометрическая нормализация. Каждый фрагмент независимо сдвинут по яркости (± 30) и контрасту (0.70-1.30). Два подхода: (1) явная **пофрагментная affine-коррекция** до сборки (оценить по статистикам краёв соседей — выровнять средние/дисперсии смежных краёв) — облегчает и матчинг, и реставрацию; (2) отдать нормализацию реставратору. **Рекомендация:** лёгкая явная нормализация ПЕРЕД матчингом (устраняет фотометрический скачок, мешающий MGC/эмбеддингам), а тонкую доводку — реставратору.

4. Часть С — Интеграция, обучение, честные трюки

Порядок операций end-to-end: пофрагментная фотометрическая нормализация → эмбеддинг (siamese) + cross-rescoring top-K → сборка (loop-constraints + Hungarian на сетке 24×24) → полнокадровое восстановление (NAFNet/Restormer+TLC) → x8 self-ensemble.

Итеративный refinement — помогает. Один-два цикла: собрать → быстро восстановить фрагменты/полукадр → пересчитать совместимости на восстановленных краях (они матчатся точнее, т.к. шум/JPEG убраны) → пересобрать. Это прямо адресует деградацию краёв: восстановленные края возвращают MGC/cross-scorer в рабочий режим.

Легитимная ТТА (не гейминг). x8 geometric self-ensemble (D4-группа: 4 поворота $\times$ 2 отражения; прогнать, инвертировать преобразование, усреднить) — стандартный приём (EDSR, NTIRE), стабильный прирост SSIM без дообучения. Легитимно и epoch/model-ensemble реставраторов (усреднение выходов). **Граница честности:** self-ensemble и усреднение моделей — легитимны (усредняют предсказание истинного сигнала); эксплойт монотонных квадратов (заливка плиток средним цветом ради ~ 0.4 SSIM) — гейминг, отвергается.

Ансамблирование. Сборка: голосование нескольких скореров (MGC + cross-scorer + siamese) в единой стоимости для Hungarian/loop. Реставрация: усреднение NAFNet и Restormer выходов (взвешенно, α подобрать на валидации), + self-ensemble.

Data strategy и label noise. Обучайте на смеси синтетика (основной объём, идеальные метки) + реальные восстановленные пары. **Проблема label noise:** $\sim 12\%$ неверно размещённых фрагментов $\rightarrow \sim 25\%$ испорченных adjacency-меток. Решения: (а) обучать скорер преимущественно на СИНТЕТИКЕ с идеальными метками (у нас известен деградатор $\rightarrow$ генерируем сколько угодно корректных пар без реконструкции); (б) hard-negative mining через near-miss (DNN-Buddies-стиль) + AdaBoost-reweighting (JigsawNet-стиль); (в) curriculum от лёгких к жёстким негативам. **Реставратор обучать почти целиком на синтетике** (7000 targets $\times$ сколько угодно деградаций), реальные пары — только для калибровки домена.

Compute budgeting (T4/P100, 8-16 ГБ). Mixed precision (AMP fp16), gradient checkpointing для Restormer. Правильно подбирайте batch под VRAM (иначе аллокатор «дёргается» и выглядит как зависание — ваш инженерный урок). На 8 ГБ — последовательное обучение модулей. Ориентиры: реставратор NAFNet width32-48, crops 240-256, batch 8-16 на T4; Restormer — batch 2-4 с checkpointing. Cross-scorer лёгкий (вход — узкая полоса шва), batch 128-256.

5. Конкретные failure modes и как их избегать

- **Неоднозначные тёмные/плоские зоны:** compatibility ненадёжна $\rightarrow$ полагайтесь на loop-constraints (4-циклы отбрасывают выбросы) и глобальное назначение; помечайте плитки с низкой дисперсией как «слабые» и решайте их в последнюю очередь.
 - **Утечка на границах фрагментов (boundary leakage):** JPEG-8 $\times$ 8-grid и швы 20-px могут давать артефакты — восстанавливать полнокадрово, TLC, при пофрагментной обработке — overlap+cosine-blend.
 - **Train-test несоответствие статистик:** TLC на инференсе.
 - **Ограничение размера сабмита PNG** (было ≤ 10 МБ до даты, затем снято): 700 PNG RGB 480 $\times$ 480 — используйте разумную PNG-компрессию (не lossy!); проверьте суммарный размер, при необходимости — оптимизация PNG без потери пикселей.
-

Часть D (ГЛАВНЫЙ ДЕЛИВЕРАБЛ) — Полная спецификация пайплайна для AI-агента

Ниже — детальная инженерная спецификация. Агент реализует код сам; спецификация задаёт КАК.

D.0. Структура репозитория и общий поток данных

```
project/
  config/          # yaml-конфиги (пути, гиперпараметры, seed)
  data/
    degrader.py    # синтетический деградатор (модуль D.1)
    dataset.py     # Dataset'ы для реставратора и скорера (D.6)
    photometric.py # фотометрическая нормализация (D.5)
  models/
    embed_net.py   # siamese CompatNet (D.2a)
    pair_net.py    # cross-scorer PairwiseNet / seam-CNN (D.2b)
    restorer.py    # NAFNet/Restormer обёртка + TLC (D.4)
  solver/
    compat.py      # построение матрицы совместимости (MGC + нейро)
    assemble.py    # loop-constraints + Hungarian на сетке 24×24 (D.3)
  infer/
    pipeline.py    # end-to-end инференс (D.7)
    tta.py         # x8 self-ensemble
  eval/
    validate.py    # локальная валидация (D.8)
    ssim.py        # SSIM win_size=7, data_range=255, channel_axis=2
  train_restorer.py
  train_scorer.py
```

Поток: raw test PNG → нарезка 576 плиток → photometric.normalize → embed_net (эмбеддинги) → compat.build_matrix (siamese top-K → pair_net rescoring + MGC) → assemble.solve (loop+Hungarian) → сборка 480×480 → restorer.restore (+TLC) → tta.self_ensemble → save PNG.

D.1. Модуль синтетического деградатора (`data/degrader.py`)

Ответственность: из чистого target PNG сгенерировать деградированные плитки, идентичные распределению реальных inputs. **Ключевая функция (концептуально):** `degrade_tile(tile: np.uint8[20,20,3], rng) -> np.uint8[20,20,3]`. **Пайплайн на плитку (точные диапазоны):**

- Affine фотометрия:** brightness += $U(-30, +30)$; contrast *= $U(0.70, 1.30)$ вокруг среднего плитки; клип $[0, 255]$.
- Гауссов шум:** добавить $N(0, \sigma^2)$, $\sigma \sim U(40, 55)$; допустить лёгкую пространственную корреляцию (наблюдался eff. std ~12–14 после блюра) — применяйте шум ДО блюра.

3. **Гауссов блюр 3×3** ($\sigma_{\text{blur}} \sim 0.5-1.0$, подобрать чтобы совпало с реальным).
4. **JPEG**: quality $\sim U(35,50)$, через реальный JPEG-кодек (cv2/PIL) или DiffJPEG для дифференцируемости. **Domain randomization (Real-ESRGAN-стиль, для робастности)**: с вероятностью ~ 0.2 менять порядок шаг2↔шаг3; расширять границы диапазонов на $\pm 10-15\%$; с вероятностью ~ 0.1 двойной JPEG. **Валидация деградатора**: сгенерировать плитки из train/targets, сравнить распределение (std шума, спектр, гистограмма яркости) с train/inputs; целевой $\Delta\text{SSIM} \leq 0.03-0.05$. **Функция полного изображения**: `(degrade_image(clean_480, rng) -> (shuffled_tiles, permutation, degraded_assembled))` — деградирует каждую из 576 плиток независимо, опционально перемешивает; возвращает и деградированную-собранную-в-правильном-порядке версию (для обучения реставратора) и перемешанную (для обучения скорера).

D.2. Модели совместимости

D.2a. Siamese CompatNet (`models/embed_net.py`) — быстрый ретривер.

- Вход: плитка $20 \times 20 \times 3$ (нормализованная). Backbone: небольшой CNN (например, 4-5 conv-блоков, GroupNorm, GELU) → вектор.
- Выход: **4 краевых эмбединга** (top/right/bottom/left), dim $d \approx 64-128$. Right-эмбединг плитки i должен совпадать (dot-product) с left-эмбедингом соседа j .
- Обучение: **симметричный InfoNCE** по всем парам через matmul (быстро). **Обязательно hard-negative mining** — включать в знаменатель near-miss (визуально похожие, но не смежные) края, а не только in-batch random; иначе recall упирается. Увеличить контекст: подавать не 1, а 2-3 крайних ряда пикселей.
- Назначение: дать top-K ($K \approx 20-50$) кандидатов на каждый край для реранжирования.

D.2b. Cross-scorer PairwiseNet / seam-CNN (`models/pair_net.py`) — точный реранкер.

- **Архитектура (early fusion, JigsawNet-стиль)**: вход — полоса шва, формируемая из смежных краёв двух фрагментов, например тензор ($C=3, H=20, W=2 \cdot w_{\text{ctx}}$) где $w_{\text{ctx}}=3-6$ крайних столбцов каждого фрагмента (ваша конфигурация (3,20,40) корректна). CNN 6-10 conv-слоёв с **вниманием, сфокусированным на стыке** (RoIAlign/центральный кроп на шве), → скаляр-логит «смежны/нет».
- **Обучение**: бинарная классификация. Позитивы — истинно смежные края (из синтетики, идеальные метки). Негативы — **hard**: (i) визуально похожие несмежные (top по siamese, но неверные), (ii) пространственно-правдоподобные, но неверные. **AdaBoost-reweighting** или focal loss против дисбаланса (JigsawNet: precision 13.7%→77.9% за счёт seam-RoI + boosting).
- **Использование**: реранжировать siamese-top-K (≈ 30 мин на 700 test — приемлемо) для каждого из 4 направлений; выход — уточнённая попарная совместимость.

- **Обоснование против siamese-потолка:** cross-encoder > bi-encoder (Humeau et al. ICLR 2020: +~3 п. R@1); DNN-Buddies precision 94.83% на конкатенированных полосах.

D.3. Солвер сборки (`(solver/)`)

`compat.build_matrix`: для каждого из 4 направлений построить матрицу $N \times N$ ($N=576$) стоимостей, комбинируя: (1) MGC по фотометрически-нормализованным (в идеале — денойзенным) краям; (2) siamese dot-product; (3) cross-scorer логит на top-K. Нормировать через **dissimilarity ratio** (Gallagher: score / second-best) и Pomeranz-преобразование $C = \exp(-D/\text{quartile})$. `assemble.solve` — эксплуатация сетки 24×24 :

1. **Best-buddies + loop-constraints (Son et al.):** найти 4-циклы, где взаимная совместимость в топе — как outlier-rejection; собрать надёжные блоки.
2. **Greedy tree-based пост** (Gallagher) от самых уверенных стыков, наращивая компоненты с проверкой геометрической согласованности на решётке.
3. **Глобальная согласованность — Hungarian:** финальное присвоение 576 фрагментов 576 ячейкам решается **венгерским алгоритмом** (`scipy.optimize.linear_sum_assignment`) по стоимости, интегрирующей все 4-направленные совместимости + частичную сборку из шагов 1-2 как жёсткие/мягкие ограничения. Опционально — Gumbel-Sinkhorn как дифференцируемый доводчик.
4. **Неоднозначные зоны:** плитки с низкой дисперсией (тёмные/плоские) решать последними, доверяясь глобальному назначению и уже зафиксированным соседям.
Выход: перестановка (`fragment_id` → (`row,col`) в сетке 24×24).

D.4. Реставратор (`(models/restorer.py)`)

- **Архитектура:** NAFNet (width 32-48, enc/dec blocks [2,2,4,8], middle 12) для бюджетного трека; Restormer (dim 48, 4 уровня) для сильного. Глобальный residual (сеть предсказывает остаток к входу). Обернуть TLC (локальная агрегация на инференсе).
- **Вход/выход:** деградированное собранное $480 \times 480 \times 3$ (uint8 → float [0,1] или [0,255]) → восстановленное $480 \times 480 \times 3$.
- **Лосс:** $L = 0.84 \cdot L_{\text{MS-SSIM}} + 0.16 \cdot (G_{\sigma} \cdot L_1) + \lambda \cdot L_{\text{SSIM}}(\text{win}=7)$. **Без adversarial/LPIPS** (perception-distortion tradeoff понижает SSIM). Charbonnier вместо чистого L1 — опционально для стабильности.
- **Обучение:** на синтетических парах (`degrade_image` собранной версии) + подмешивание реальных восстановленных. Crops 240-256, random flip/rot (согласуется с self-ensemble). AMP, grad checkpointing (Restormer). Optimizer AdamW, cosine LR, ~200-400k итераций.
- **Решение по seam:** восстанавливать ПОЛНОЕ изображение (сеть чинит швы). Пофрагментный вариант — только вспомогательный, с overlap+cosine-blend.

D.5. Фотометрическая нормализация ((data/photometric.py))

- `normalize_tiles(tiles) -> tiles_norm`: оценить per-tile affine (mean/std) и выровнять к глобальной статистике или к статистикам смежных краёв (после предварительной черновой сборки — итеративно). Лёгкая коррекция ПЕРЕД матчингом. Тонкую доводку оставить реставратору.

D.6. Датасеты ((data/dataset.py))

- `RestorerDataset`: берёт clean 480×480 → `degrade_image` → (`degraded_assembled`, clean) пары; аугментации flip/rot.
- `ScorerDataset`: берёт clean → нарезает плитки → `degrade_tile` каждую → выдаёт (edge_i, edge_j, label) с hard-negative sampling; для siamese — батчи с in-batch + mined negatives; для pair_net — seam-тензоры с позитив/жёсткий-негатив балансом.

D.7. End-to-end инференс ((infer/pipeline.py))

Точный порядок на один test PNG:

1. Загрузить деградированное 480×480; нарезать на 576 плиток 20×20 (по grid 24×24).
2. `photometric.normalize_tiles`.
3. `embed_net` → 4 эмбединга/плитку; `compat`: siamese top-K → `pair_net` rescoring → комбинированная матрица (+MGC).
4. `assemble.solve` → перестановка → собрать 480×480 в правильном порядке.
5. **(Опционально) итеративный refinement**: быстрый прогон restorer на собранном → пересчёт совместимостей на восстановленных краях → пересборка (1 цикл).
6. `restorer.restore` (полнокадрово, +TLC).
7. `tta.self_ensemble` — x8 D4: прогнать restorer на 8 преобразованиях, инвертировать, усреднить.
8. Клип [0,255], uint8, сохранить `img_XXXXXX.png` (RGB, 480×480, PNG без потерь).

D.8. Локальная валидация ((eval/validate.py))

- Держать последние 300 (из 7000) train как hold-out; для них известны targets.
- **Восстановление GT-расстановки**: сопоставить деградированные плитки hold-out с плитками из target через **Hungarian на дескрипторах** (как вы уже делаете) → получить ground-truth перестановку для измерения точности сборки БЕЗ реставрации.
- **Раздельные метрики**: (a) placement accuracy = доля верно размещённых фрагментов (direct comparison metric) и neighbor-accuracy; (b) restoration SSIM при ИДЕАЛЬНОЙ расстановке (изолирует реставратор); (c) end-to-end mean SSIM (win_size=7, data_range=255, channel_axis=2). Так каждый модуль измеряется независимо.

D.9. Формат сабмита

- 700 PNG, RGB, 480×480, имена `img_XXXXXX.png`, плоский zip (без вложенных папок). PNG lossless. Проверить суммарный размер против лимита (был ≤10 МБ до снятия) — при необходимости оптимизировать PNG (уровень компрессии) без потери пикселей.

D.10. Порядок реализации и milestone'ы (независимо валидируемые)

1. **M1 — деградатор + валидация.** Реализовать `degrader.py`, `eval/ssim.py`, `validate.py`. Критерий: синтетика $\Delta\text{SSIM} \leq 0.05$ к реальным inputs. (Разблокирует всё обучение.)
2. **M2 — реставратор (крупнейший рычаг).** Обучить NAFNet на синтетике; мерить restoration-SSIM при идеальной расстановке (метрика b). Критерий: SSIM 0.6–0.8 на hold-out с GT-расстановкой. Это уже даёт сабмит, бьющий лидера при любой разумной сборке.
3. **M3 — siamese CompatNet + Hungarian baseline.** Мерить placement accuracy (метрика a). Критерий: заметно поднять recall@1 hard-negative-майнингом над 0.16.
4. **M4 — cross-scorer PairwiseNet (rescoring).** Реранжировать top-K. Критерий: neighbor-accuracy → 0.9+.
5. **M5 — loop-constraints + глобальное назначение.** Критерий: near-perfect placement на чистых, робастность на деградированных.
6. **M6 — итеративный refinement + x8 self-ensemble + ансамбль реставраторов.** Критерий: прирост end-to-end SSIM.
7. **M7 — сильный трек:** Restormer+TLC, ансамбль, финальная калибровка.

D.11. Известные ловушки (из ваших инженерных уроков) и как избежать

- **VRAM-thrashing выглядит как зависание:** правильно размеряйте batch под GPU; на 8 ГБ — последовательное обучение модулей, grad checkpointing.
- **Label noise ~25% в реальных adjacency:** обучайте скореры на синтетике с идеальными метками; реальные пары — только для домен-калибровки.
- **Siamese-потолок:** не полагайтесь на dot-product для точного ранжирования — используйте retrieve-then-rerank с cross-scorer.
- **SSIM vs честность:** не поддавайтесь на adversarial/LPIPS ради «красоты» — они понизят SSIM; MS-SSIM+L1+SSIM(win7) — и честно, и оптимально под метрику.
- **Швы:** полнокадровая реставрация + TLC; пофрагментно — только с overlap+blend.

Рекомендации (staged, с порогами переключения)

1. **Немедленно (M1–M2):** зафиксировать деградатор (порог $\Delta\text{SSIM} \leq 0.05$) и обучить NAFNet-реставратор. Это самый быстрый путь обойти лидера (0.40): при restoration-

SSIM $\geq$ 0.6 на GT-расстановке даже средняя сборка даёт итог выше лидера. **Порог переключения:** если restoration-SSIM $<$ 0.55 — расширяйте синтетику (domain randomization, двойной JPEG), проверяйте калибровку деградатора.

2. **Далее (M3–M5):** поднимать сборку. **Порог:** если placement accuracy siamese-only $<$ 0.7 — обязательно добавляйте cross-scorer (M4); если neighbor-accuracy $<$ 0.9 — включайте loop-constraints + Hungarian (M5). Если и это не даёт near-perfect на чистых — увеличивайте контекст шва и hard-negative-майнинг.
3. **Оптимизация (M6–M7):** итеративный refinement (порог: даёт $\geq +0.005$ SSIM — оставить), x8 self-ensemble (почти всегда +), ансамбль NAFNet+Restormer. Переходить на Restormer+TLC на сильном треке, если бюджет GPU позволяет batch ≥ 2 с checkpointing.
4. **Постоянно:** мерить три метрики (placement, restoration@GT, end-to-end) отдельно на hold-out 300 — это единственный способ понять, какой модуль тормозит.

Предостережения

- **Числа калибровки** (0.6–0.8 для сборка+реставрация) — ваши эмпирические оценки; реальный потолок зависит от узости домена и точности деградатора. Перепроверяйте restoration@GT на hold-out.
- **Δ SSIM $\approx$ 0.03** синтетики к реальной — приятно мало, но остаточный domain gap может стоить 0.01–0.03 итогового SSIM; подмешивание реальных восстановленных пар и domain randomization критичны.
- **Perception-distortion tradeoff** — строгий теоретический результат (Blau & Michaeli 2018): любая попытка «улучшить визуально» через генеративные лоссы под метрикой SSIM ухудшит счёт. Это ограничивает «честность» рамками regression-to-mean реставрации — что для данной метрики и есть оптимум. (TheCVF)
- **Retrieve-then-rerank стоимость:** cross-scorer на всех N^2 дорог (~30 мин/700 изображений по вашим замерам); держите его только на top-K, иначе инференс не уложится в бюджет.
- **Robustность эвристик к коррупции** (Dirauf et al.) подтверждает: без обучаемого скорера классические MGC/L1 на деградированных краях просядут — не полагайтесь на них в одиночку.
- **Формат/лимит сабмита** менялся во времени (≤ 10 МБ→снят) — сверяйтесь с актуальными правилами соревнования перед финальной отправкой.